Draft version February 27, 2026
Typeset using LaTeX **twocolumn** style in AASTeX7

Homework[3]: Numerical Interpolation

Haikun Xue[1]

[1] *University of Oklahoma, Department of Astronomy*

1. LINKS TO CODE

Here is a link to the code file https://github.com/HaikunXue/OU-ASTR-5900/blob/main/HW3/HW03.ipynb

2. QUESTION 1:

2.1. *Part a:*

We have the derivative equation:

$$\frac{dy}{dx} = y^2 + 1$$

Figure 1 shows the initial attempt of fitting over the range of $\{0, 2\pi\}$ with 100 steps. Runge-Kutta method has a better initial fit, Euler's method undershoot the function, but both functions trend towards infinity around $x = \pi/2$, where we know that $\tan(\pi/2) \approx \infty$.

So, we will evaluate the function at a smaller range, within $0, \pi/2$, as shown in Figure 2, mid pannel.

2.2. *Part b:*

As shown in Figure 1, both method breaks at the point of $\pi/2$. We know that at that point, the function $\tan(x) \approx \infty$. In addition, the derivative, $\frac{dy}{dx} = y^2 + 1$, leads to an exponential increase beyond y = 1, and that's when the interpolation breaks. The exact break point is shown in table 1.

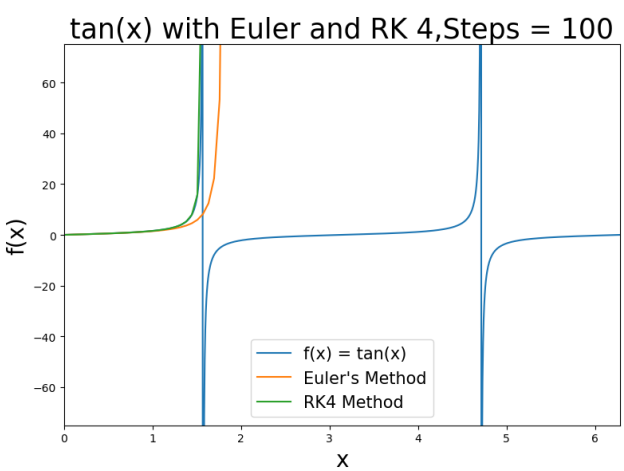

Figure 1. Initial attempt over the range $\{0, 2\pi\}$. Blue line is tan(x), orange line is Euler's method, and green line is RK4.

x position	Euler	RK-4
1.57	8.17×10^{0}	1.31×10^{2}
1.63	1.24×10^{1}	1.64×10^{12}
1.70	2.22×10^{1}	1.05×10^{173}
1.76	5.33×10^{1}	∞
1.82	2.32×10^{2}	∞
1.88	3.60×10^{3}	∞
1.95	8.18×10^{5}	∞
2.01	4.20×10^{10}	∞
2.07	1.11×10^{20}	∞
2.14	7.74×10^{38}	∞
2.20	3.77×10^{76}	∞
2.26	8.91×10^{151}	∞
2.32	4.98×10^{302}	∞
2.39	∞	∞
2.45	∞	∞

Table 1. Euler vs. RK-4 values for indices $i = 25$–39. Euler reaches breaking limit much slower than RK-4

2.3. *Part c:*

We did some testing with different step values, shown in Figure 2. Compares with the 100 steps graph earlier, we can see that RK-4 fit the actual function very well, even at n= 10, while Euler's method barely catches up at n= 1000.

2.4. *Part d:*

With the convergent study, we use $n = 10^7$ steps as our best guess, and compares to previous results with

$$\frac{best - current}{best}$$

to plot Figure 3, which results in a slope 0.868 for Euler's method, and 3.806 for RK-4 method. This is inline with the expectation that Euler is a first order method, and RK-4 is a fourth order method. Of note, RK4's method cuts off at 10^{-4}, upon future inspection, the lower step size lead to positive difference between endpoint value

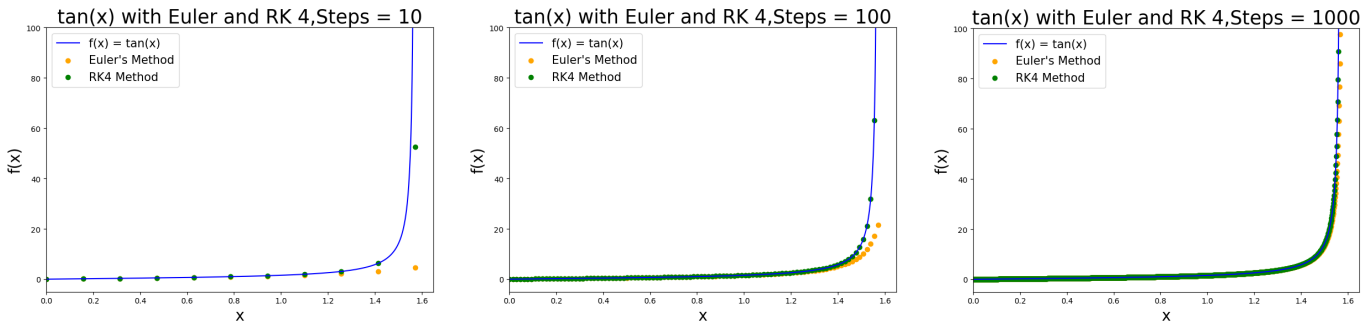

Figure 2. A more well informed attempt over the range $\{0, \pi/2\}$. Blue line is tan(x), orange line is Euler's method, and green line is RK4. Left: Steps = 10, RK 4 is well fitted; Mid: Steps = 100, Euler is converging to the line; Right: Steps = 1000, both are well fitted.

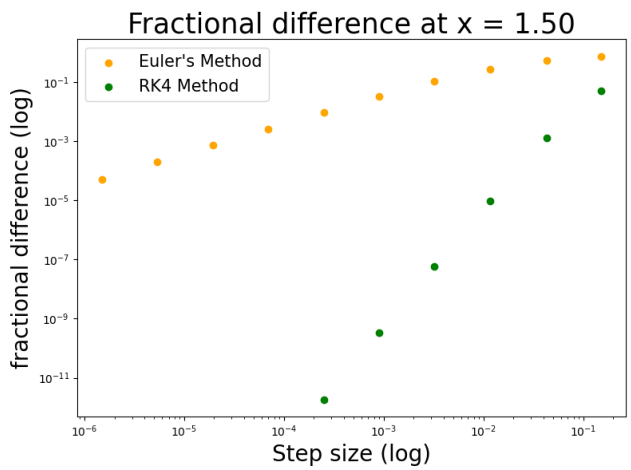

Figure 3. Convergence study case 1

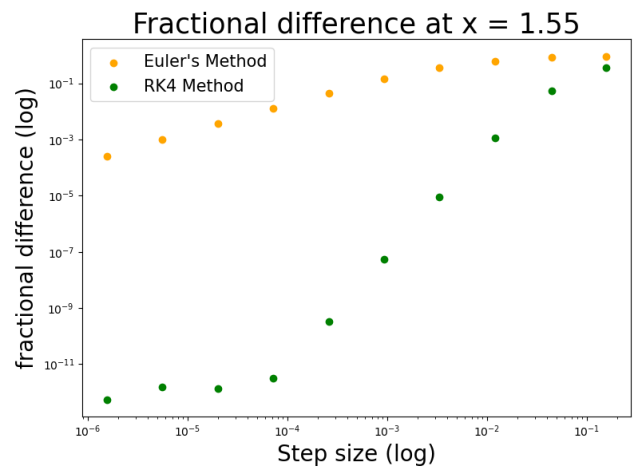

Figure 4. Convergence study case 2

and best guess, which implies it reaches computational limit for the code.

In comparison, when choose x = 1.55, there are more variation of the curve, and we see the similar fluctuation in RK-4, beyond 10^{-4}. In this case, for the whole sample size, including the fluctuations, we get 0.74 for Euler and 2.68 for RK-4. This could be where the algorithms breaks!

2.5. *Part e:*

With python, the actual value at x = 1.5 is :

$$\tan(1.5) = 14.1014$$

Our best guess for Euler is 14.1013, which lead to a fractional difference: 5.6309×10^{-6}. For our two best guess, the fractional difference: 5.0674×10^{-5}, so it is off by a order of magnitude.

Our best guess for RK-4 14.1014, which lead to a fractional difference: 2.0961×10^{-13}. For our two best guess, the fractional difference: 3.8236×10^{-13}, so it is very close. Keep in mind this is reaching computational limits

3. QUESTION 2

Our fraction $f(v)$ is:

$$f(v)dv = [\frac{m}{2\pi kT}]^{\frac{3}{2}} 4\pi v^2 \exp(-\frac{mv^2}{2kT})dv$$

3.1. *Part a:*

We plotted the probability density distribution by velocity, shown in Figure 5. Adding up areas under the curve, we get 0.9899, for 100 steps, which is close to 100%. With 10000 steps, we get 0.99989, which is much closer to 1, and a smoother curve.

3.2. *Part b:*

To calculate the minimum excited velocity for Hydrogen to reach the first excited state, we use:

$$KE = -\frac{13.6}{n^2}$$

to find the required energy, then use

$$KE = \frac{1}{2}mv^2$$

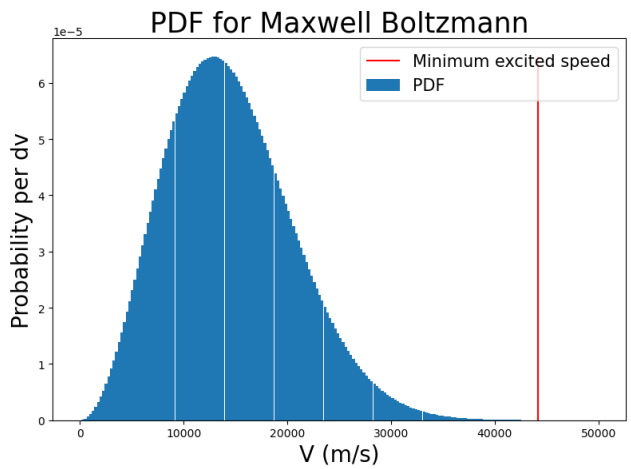

Figure 5. Probability density distribution for hydrogen atom

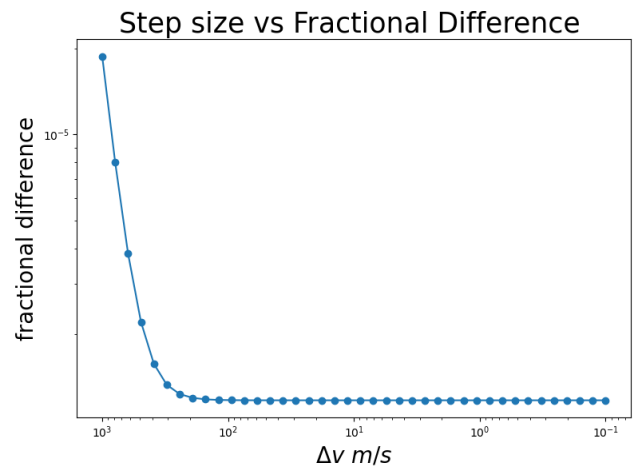

Figure 7. Step sizes vs fractional difference, comparing to our best approach from approach 1.

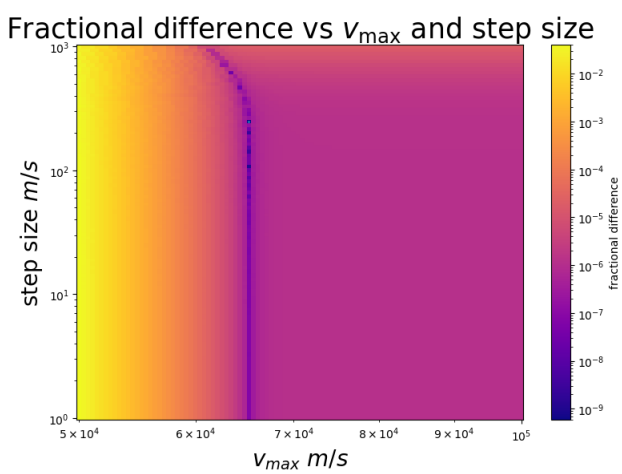

Figure 6. Step sizes vs maximum v value vs fractional difference, comparing to our best approach from approach 1. All three axis are in log scale, and evenly scaled.

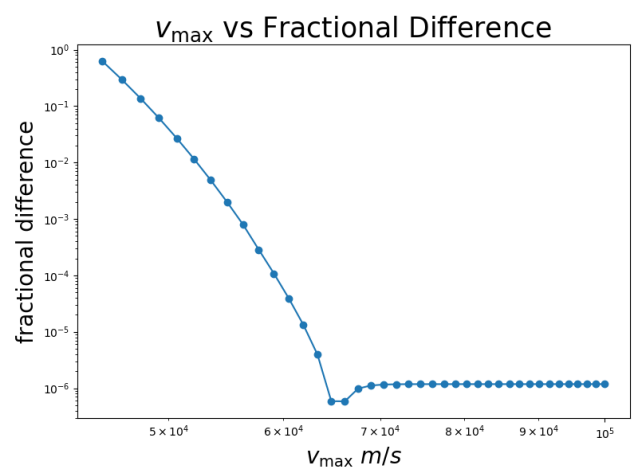

Figure 8. v max vs fractional difference, comparing to our best approach from approach 1.

to find the velocity $v = 44188.32 m/s$, as shown in the redline in Figure 5. So only a small fraction of hydrogen atoms will escape.

We approach the fraction of hydrogen atoms that would escape with two approaches.

First, we integrate from $v = 0$ to $v_{min} = 44188$, to get the percentage of points *not* excited. With 10^7 steps, we get the $2.9252759 \times 10^{-03}\%$ of atoms. For 10^6 steps, we get $2.9252794 \times 10^{-03}\%$ of atoms, so the result is accurate up til 10^{-9}. We will use the 10^7 as our "actual" value for comparison for the second approach.

In the second approach, we integrate from $v_{min} = 44188$ to "∞", with various step sizes and various "∞" values, as shown in Figure 6. We can see that the fractional difference changes both with v_{max}, and step size, with a interesting gap at 70,000 m/s.

Then we took closer look at each axis. For step size variations, shown in Figure 7. With a v_{max} of 100,000, We see the integration algorithm plateau at about 200, and stabilize around step size of 10 of 10^{-6} error. This could be due to round-off errors.

In comparison, for v_{max} variation, which is shown in Figure 8. With a step size of 100, there is drop at 65,000 m/s, then stabilize to a error of 10^{-6} after 80,000 m/s. This could mean that we account for most of the data for the tail of the distribution before 80,000 m/s due to our computational limit.

3.3. *Part c:*

The best bet of v_{max} is around 80,000, and the best dv is around 10. With those two, our integration routine takes 10.6 $ms \pm 319$ μsms (on my computer) and has a fractional difference of order 10^{-6}, which will lead to an

error bar of order 10^{-11}. This is a reasonable result for our purpose.

ACKNOWLEDGMENTS

Thanks to Professor Sean Matt for the template.